



## **Chapter S3B: Relational Database**

### **CONTENT**

| <b>No.</b> | <b>Topic</b>                       | <b>Syllabus Guide</b> |
|------------|------------------------------------|-----------------------|
| <b>1</b>   | <b>Database</b>                    |                       |
| <b>2</b>   | <b>Flat file</b>                   |                       |
| <b>3</b>   | <b>Relational database</b>         | <b>3.3.1</b>          |
| <b>3.1</b> | <b>Candidate key</b>               | <b>3.3.2</b>          |
| <b>3.2</b> | <b>Primary key</b>                 |                       |
| <b>3.3</b> | <b>Secondary key</b>               |                       |
| <b>3.4</b> | <b>Composite key</b>               |                       |
| <b>3.5</b> | <b>Foreign key</b>                 |                       |
| <b>3.6</b> | <b>Entity-relationship diagram</b> | <b>3.3.5</b>          |
| <b>4</b>   | <b>Data redundancy</b>             | <b>3.3.3</b>          |
| <b>5</b>   | <b>Data dependency</b>             | <b>3.3.3</b>          |
| <b>5.1</b> | <b>Functional dependency</b>       |                       |
| <b>5.2</b> | <b>Transitive dependency</b>       |                       |
| <b>6</b>   | <b>Normalisation</b>               | <b>3.3.4</b>          |
| <b>6.1</b> | <b>First normal form</b>           |                       |
| <b>6.2</b> | <b>Second normal form</b>          |                       |
| <b>6.3</b> | <b>Third normal form</b>           |                       |

## 1. DATABASE

A **database** is a collection of data items and links between them, structured so it can be accessed by different applications. It provides a system for storing, accessing, and managing data.

The term is sometimes used loosely to describe any data collection. Examples of real-life databases include:

- Library catalogues
- Patient medical records
- Inventory management systems
- Airline reservation systems
- Human resource management systems

Benefits of databases include:

- Data integrity
- Data security
- Efficient data retrieval
- Support for complex queries and analysis

In this course, we will use **SQL** (Structured Query Language) and **DB Browser** to manage and interact with databases.

## 2. FLAT FILE

At its simplest, a database can be just a **flat file** – a single table stored in a text file.

- Each row is a *record*.
- Each column is a *field*.

This can be visualised as a two-dimensional table (hence the term “flat”).

**Example:**

|                 | <i>Field 1</i> | <i>Field 2</i> | <i>Field 3</i>   |
|-----------------|----------------|----------------|------------------|
|                 | <b>Name</b>    | <b>Address</b> | <b>Telephone</b> |
| <i>Record 1</i> | Glen           | 8 ABC Road     | 64532302         |
| <i>Record 2</i> | Kenneth        | 34 XYZ Ave     | 65304859         |
| <i>Record 3</i> | Mohan          | 738 KDK St     | 65638704         |
| <i>Record 4</i> | Yang Yang      | 495 ZHS Lane   | 67738294         |
| <i>Record 5</i> | ... ..         | ... ..         | ... ..           |

Flat files are easy to create and work well for small-scale applications. Application programs can be written to access them using any programming language.

As data grows and relationships become more complex, flat files become harder to maintain. Managing links between data and performing advanced queries can be inefficient. This is where **relational databases** help as they organise data into related tables, ensuring data integrity and enabling efficient queries.

## Differences between Flat-file database & Relational database

| Flat-file database                                                                                                                                                                                                | Relational database                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data is stored in a single table or file.<br><br><b>Example:</b> A CSV file containing all customer names, addresses, and orders in one sheet.                                                                    | Data is stored in multiple tables (relations).<br><br><b>Example:</b> A “Customers” table, an “Orders” table, and a “Products” table, each linked by keys.                                                                                                                 |
| No built-in relationships between data sets; links must be created manually.<br><br><b>Example:</b> If a customer moves, you must manually find and change their address in every file where it appears.          | Relationships between tables are established using keys (primary and foreign keys).<br><br><b>Example:</b> Change the address once in the “Customers” table and all linked orders automatically reference the updated address.                                             |
| Changes to structure require manual updates; no automatic enforcement of data constraints.<br><br><b>Example:</b> Adding a new field like “Email” means editing the file structure and all programs that read it. | Structure changes and constraints are handled by the relational model; updates in one place are reflected throughout the database.<br><br><b>Example:</b> Adding an “Email” column to the “Customers” table automatically applies to all linked queries and relationships. |
| Often requires custom programs or scripts to retrieve specific information.<br><br><b>Example:</b> Writing a Python script to search for all orders above \$500.                                                  | Query languages like SQL allow easy data retrieval and manipulation.<br><br><b>Example:</b> Using <code>SELECT * FROM Orders WHERE Total &gt; 500;</code> .                                                                                                                |
| Higher risk of data redundancy and inconsistency.<br><br><b>Example:</b> Same customer’s details are copied into multiple order records, leading to mismatched data if one copy changes.                          | Reduces redundancy and improves consistency through normalisation.<br><br><b>Example:</b> Customer details stored once in “Customers” table and referenced by “Orders” table.                                                                                              |

Flat-file databases are simple and work well for small, single-purpose datasets. However, they become inefficient and error-prone as data grows in size and complexity. Relational databases organise data into linked tables, reducing duplication and improving accuracy with keys and constraints. This structure makes it easier to update, maintain, and retrieve data using SQL, and provides greater flexibility, consistency, and scalability for larger, interconnected datasets.

### 3. RELATIONAL DATABASE

In the relational database model, data is stored in relations and represented in the form of tuples (rows). A relational database is a collection of relational tables.

A table is a two-dimensional representation of data stored in rows and columns.

A table description can be expressed as:

TableName (Attribute1, Attribute2, Attribute3, ...)

For example,

Student (RegNo, Name, Gender, CivicsClass, CivicsTutor)

A table is a relation if it fulfils the following conditions:

- Values are atomic
- Columns are of the same kind
- Rows are unique
- The order of columns is insignificant
- Each column must have a unique name

An example of a table Student28S18, showing data for students from civics class 28S18:

| Field |        |        |          |
|-------|--------|--------|----------|
| RegNo | Name   | Gender | MobileNo |
| 1     | Adam   | M      | 92313291 |
| 2     | Adrian | M      | 92585955 |
| 3     | Agnes  | F      | 83324112 |
| 4     | Aisha  | F      | 88851896 |
| 5     | Ajay   | M      | 94191061 |
| 6     | Alex   | M      | 98675171 |
| 7     | Alice  | F      | 95029176 |
| 8     | Amy    | F      | 98640883 |
| 9     | Andrew | M      | 95172444 |
| 10    | Andy   | M      | 95888639 |

Tables are made up of records and fields.

A **record** is a complete set of data about a single item. For this example, a record refers to the complete set of data of a particular student.

A field holds a single, atomic value within a record. It represents a specific attribute or property of the data and is associated with a column in a table.

**Quiz:** How many records does the table Student28S18 have?

#### 3.1 Candidate Key

A **candidate key** is defined as a minimal set of fields which can uniquely identify each record in a table. It tells a particular record apart from another record. A candidate key should never be NULL or empty. There can be more than one candidate key for a table. A candidate key can also be a combination of more than one field.

**Quiz:** Which of the fields in table Student28S18 are candidate keys?

#### 3.2 Primary Key

A **primary key** is a candidate key that is most appropriate to become the main key for a table. It uniquely identifies each record in a table and should not change over time. That is, a primary key tells a particular record apart from another record.

**Quiz:** Which of the candidate key is a suitable primary key for the table Student28S18?

### 3.3 Secondary Key

Candidate keys that are not selected as primary key are known as secondary keys.

### 3.4 Composite Key

Sometimes, more than one field is needed to uniquely identify a record. A **composite key** is a combination of two or more fields in a table that can be used to uniquely identify each record in a table. Uniqueness is only guaranteed when the fields are combined. When taken individually, the fields do not guarantee uniqueness.

Take a look at the table `Student` below.

| RegNo | Name              | Gender | CivicsClass |
|-------|-------------------|--------|-------------|
| 1     | Aisyah Bte Rahman | Female | 28S28       |
| 2     | Cheryl Lim        | Female | 28S28       |
| 3     | Daniel Goh        | Male   | 28S28       |
| 4     | Farhan Yusof      | Male   | 28S28       |
| 5     | Marcus Tan        | Male   | 28S28       |
| 6     | May Tay           | Female | 28S28       |
| 7     | Vikram Nair       | Male   | 28S28       |
| 1     | Arjun Singh       | Male   | 28S30       |
| 2     | Benjamin Ong      | Male   | 28S30       |
| 3     | Huda Ismail       | Female | 28S30       |
| 4     | Mei Ling          | Female | 28S30       |
| 5     | Nurul Aini        | Female | 28S30       |
| 6     | Ryan Pereira      | Male   | 28S30       |
| 7     | Shanti Devi       | Female | 28S30       |

Now, a single field is not able to uniquely identify a record. A composite key composing 2 fields is needed to uniquely identify a record.

**Quiz:** Which 2 fields would you choose to form a composite key for the table `Student`?

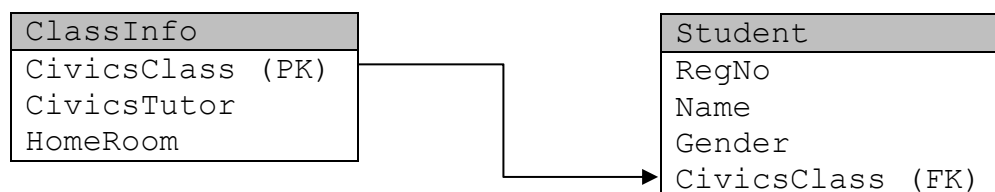
### 3.5 Foreign Key

Another table, `ClassInfo`, as shown below, stores the information about each civics class.

| CivicsClass | CivicsTutor | HomeRoom |
|-------------|-------------|----------|
| 28S28       | Paul Poh    | A33      |
| 28S30       | David Lee   | B55      |

`CivicsClass` is chosen to be the primary key (PK) for table `ClassInfo`.

Notice that the primary key in table `ClassInfo` (i.e., the `CivicsClass` field) is related or linked to the `CivicsClass` field in table `Student`. This makes `CivicsClass` field in table `Student` a foreign key (FK).



A **foreign key** is an attribute (field) in one table **that refers to the primary key in another table**. The foreign key can be used to cross-reference tables. Therefore, foreign key cannot contain values that **do not appear** in the table that it refers to.

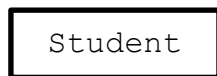
**Primary key does not allow duplicate and null values, but foreign key allows duplicate and null values.**

### 3.6 Entity-Relationship Diagram

Designers of a large database will normally construct a diagram of the planned database. An example of this is entity-relationship (E-R) diagram.

An **entity** is a specific object of interest. Collective nouns or nouns are usually used to name entities. Entities are represented by rectangles.

For example,



**Relationship** describes a link between two entities.

One of the following three relationships can exist between two entities.

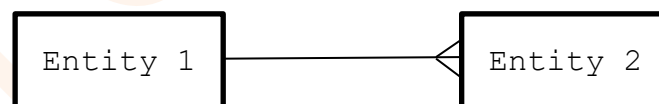
#### 1. One to One



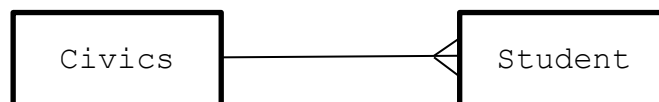
For example, at a concert each ticket entitles you to a particular seat and each seat is linked to only one ticket.



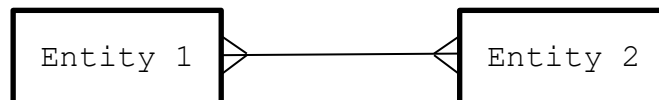
#### 2. One to Many



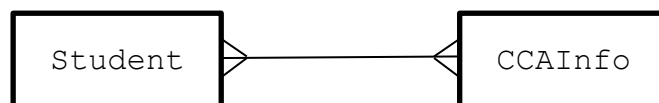
For example, a student can belong to only one civics class but a civics class can have many students.



#### 3. Many to Many



For example, a student can join many CCAs and one CCA can have many students.



## 4. DATA REDUNDANCY

**Data redundancy** refers to the same data being stored more than once. It is a data organisation issue that allows the unnecessary duplication of data within the database.

Look at this example of `Student` table.

| RegNo | Name              | Gender | CivicsClass | CivicsTutor |
|-------|-------------------|--------|-------------|-------------|
| 1     | Aisyah Bte Rahman | Female | 28S28       | Paul Poh    |
| 2     | Cheryl Lim        | Female | 28S28       | Paul Poh    |
| 3     | Daniel Goh        | Male   | 28S28       | Paul Poh    |
| 4     | Farhan Yusof      | Male   | 28S28       | Paul Poh    |
| 5     | Marcus Tan        | Male   | 28S28       | Paul Poh    |
| 6     | May Tay           | Female | 28S28       | Paul Poh    |
| 7     | Vikram Nair       | Male   | 28S28       | Paul Poh    |

As we can see, data for `CivicsClass` and `CivicsTutor` are repeated for students who are in the same civics class. This is data redundancy.

This will cause issues when inserting, updating and deleting data from the database.

|                          |                                                                                                                                                                                                                                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Insertion anomaly</b> | Occurs when data cannot be added to the database because other required data is missing.<br><br><b>Example:</b> A new student cannot be added unless a Civics Class and a Civics Tutor have already been assigned.                                                                               |
| <b>Update anomaly</b>    | Happens when changes to duplicated data are only partially made, leading to inconsistencies.<br><br><b>Example:</b> If the Civics Tutor <i>Paul Poh</i> leaves the college, every student record referencing him must be updated. If even one record is missed, it results in inconsistent data. |
| <b>Deletion anomaly</b>  | Arises when deleting a record unintentionally removes valuable data that should be retained.<br><br><b>Example:</b> If all student records are deleted, information about the Civics Class and its Tutor may also be lost.                                                                       |

## 5. DATA DEPENDENCY

To normalise data effectively, it is essential to understand the underlying data dependencies, particularly *functional* and *transitive* dependencies.

### 5.1 Functional Dependency

Attribute Y is **functionally dependent** on attribute X (usually the primary key), if for every valid instance of X, the value of X uniquely determines the value of Y ( $X \rightarrow Y$ ).

For example, suppose we have a `Student` table with the following attributes: `MatricNo`, `Name`, `Gender`, `CivicsClass` and `CivicsTutor`. `MatricNo` is a unique number assigned to every student in the college.

MatricNo uniquely identifies the Name because if we know the MatricNo, we can know the Name associated with it. Therefore, we can say Name is functionally dependent on MatricNo ( $\text{MatricNo} \rightarrow \text{Name}$ ).

## 5.2 Transitive Dependency

A functional dependency is said to be **transitive** if it is indirectly formed by two functional dependencies. Z is transitively dependent on X if Y is functionally dependent on X but X is not functionally dependent on Y, and Z is functionally dependent on Y. In other words,  $X \rightarrow Z$  is a transitive dependency if the following hold true:

- $X \rightarrow Y$
- Y does not  $\rightarrow X$
- $Y \rightarrow Z$

For example, CivicsClass is functionally dependent on MatricNo ( $\text{MatricNo} \rightarrow \text{CivicsClass}$ ) but MatricNo is not functionally dependent on CivicsClass. On the other hand, CivicsTutor is functionally dependent on CivicsClass ( $\text{CivicsClass} \rightarrow \text{CivicsTutor}$ ). Therefore, CivicsTutor is transitively dependent on MatricNo ( $\text{MatricNo} \rightarrow \text{CivicsTutor}$ ).

## 6. NORMALISATION

Normalisation is the process of organising the tables in a database to reduce data redundancy and prevent inconsistent data. Tables should be organised in a way that

- no data is unnecessarily duplicated (i.e. the same data held on more than one table).
- data is consistent throughout the database (no update anomaly).
- the structure of each table is flexible enough to allow you to enter as many or as few items as you want to (no insertion anomaly).
- the structure should enable a user to make all kinds of complex queries relating data from different tables.

There are at least three normal forms associated with normalisation: first normal form (1NF), second normal form (2NF), and third normal form (3NF).

### 6.1 First Normal Form

For a table to be in 1NF, all columns must be atomic. This means there can be no multi-valued columns – i.e., columns that would hold a collection such as an array or another table. In other words, the information in each column cannot be broken down further.

Consider the following table:

| MatricNo | Name              | Gender | CivicsClass | CivicsTutor | HomeRoom | CCAInfo                                                               |
|----------|-------------------|--------|-------------|-------------|----------|-----------------------------------------------------------------------|
| 1        | Aisyah Bte Rahman | Female | 28S28       | Paul Poh    | A33      | Tennis Teacher IC = Adrian Tan                                        |
| 2        | Cheryl Lim        | Female | 28S28       | Paul Poh    | A33      | Choir Teacher IC = Ashley Wong, AVA Teacher IC = Sandra Ho            |
| 3        | Arjun Singh       | Male   | 28S30       | David Lee   | B55      | Rugby Teacher IC = Zach Teo                                           |
| 4        | Benjamin Ong      | Male   | 28S30       | David Lee   | B55      | Badminton Teacher IC = Oh Sim                                         |
| 5        | Huda Ismail       | Female | 28S30       | David Lee   | B55      | Choir Teacher IC = Ashley Wong, Student Council Teacher IC = Hong Tee |



It is assumed that every civics class only has one civics tutor, and each CCA has only one teacher in-charge (IC).

This table is not in 1NF because the CCAInfo column contains multiple values.

In order for the table to be in 1NF, split the CCAInfo into two single-value columns CCAName and CCATeacherIC. Notice that the students with MatricNo 2 and MatricNo 5 have multiple CCAs. We keep this information intact by splitting their records into multiple records, each corresponding to a different CCA.

| MatricNo | Name              | Gender | CivicsClass | CivicsTutor | HomeRoom | CCAName         | CCATeacherIC |
|----------|-------------------|--------|-------------|-------------|----------|-----------------|--------------|
| 1        | Aisyah Bte Rahman | Female | 28S28       | Paul Poh    | A33      | Tennis          | Adrian Tan   |
| 2        | Cheryl Lim        | Female | 28S28       | Paul Poh    | A33      | Choir           | Ashley Wong  |
| 2        | Cheryl Lim        | Female | 28S28       | Paul Poh    | A33      | AVA             | Sandra Ho    |
| 3        | Arjun Singh       | Male   | 28S30       | David Lee   | B55      | Rugby           | Zach Teo     |
| 4        | Benjamin Ong      | Male   | 28S30       | David Lee   | B55      | Badminton       | Oh Sim       |
| 5        | Huda Ismail       | Female | 28S30       | David Lee   | B55      | Choir           | Ashley Wong  |
| 5        | Huda Ismail       | Female | 28S30       | David Lee   | B55      | Student Council | Hong Tee     |

The values for CCAName and CCATeacherIC are now atomic for each row. The primary key for the above table will be the composite key formed by MatricNo and CCAName.

Alternatively, the table can be split into two with entities Student and CCAInfo.

Student

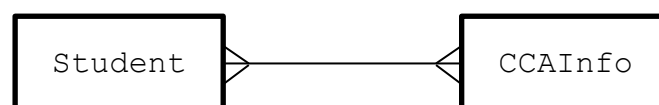
| MatricNo | Name              | Gender | CivicsClass | CivicsTutor | HomeRoom | CCAName         |
|----------|-------------------|--------|-------------|-------------|----------|-----------------|
| 1        | Aisyah Bte Rahman | Female | 28S28       | Paul Poh    | A33      | Tennis          |
| 2        | Cheryl Lim        | Female | 28S28       | Paul Poh    | A33      | Choir           |
| 2        | Cheryl Lim        | Female | 28S28       | Paul Poh    | A33      | AVA             |
| 3        | Arjun Singh       | Male   | 28S30       | David Lee   | B55      | Rugby           |
| 4        | Benjamin Ong      | Male   | 28S30       | David Lee   | B55      | Badminton       |
| 5        | Huda Ismail       | Female | 28S30       | David Lee   | B55      | Choir           |
| 5        | Huda Ismail       | Female | 28S30       | David Lee   | B55      | Student Council |

CCAInfo

| CCAName         | CCATeacherIC |
|-----------------|--------------|
| Tennis          | Adrian Tan   |
| Choir           | Ashley Wong  |
| AVA             | Sandra Ho    |
| Rugby           | Zach Teo     |
| Badminton       | Oh Sim       |
| Student Council | Hong Tee     |

The values for all columns in both tables are all atomic, making the tables in 1NF.

The two entities Student and CCAInfo have a many-to-many relationship and linked by the CCAName field. It can be represented by the entity-relationship diagram.



## 6.2 Second Normal Form

For a table to be in 2NF, it must satisfy two conditions:

- 1) The table should be in 1NF
- 2) Every non-key attribute must be fully dependent on the entire primary key. This means no attribute can depend on part of the primary key only

**A table in 2NF contains no partial dependencies.**

In the Student table on the previous page, the composite key is made up of MatricNo and CCAName. However, Name, Gender, CivicsClass, CivicsTutor and HomeRoom is dependent on only *part of the primary key*, MatricNo, while CCATeacherIC is dependent only on CCAName.

Thus, decompose into three tables to fulfil 2NF:

Student

| MatricNo | Name              | Gender | CivicsClass | CivicsTutor | HomeRoom |
|----------|-------------------|--------|-------------|-------------|----------|
| 1        | Aisyah Bte Rahman | Female | 28S28       | Paul Poh    | A33      |
| 2        | Cheryl Lim        | Female | 28S28       | Paul Poh    | A33      |
| 3        | Arjun Singh       | Male   | 28S30       | David Lee   | B55      |
| 4        | Benjamin Ong      | Male   | 28S30       | David Lee   | B55      |
| 5        | Huda Ismail       | Female | 28S30       | David Lee   | B55      |

CCAInfo

| CCAName         | CCATeacherIC |
|-----------------|--------------|
| Tennis          | Adrian Tan   |
| Choir           | Ashley Wong  |
| AVA             | Sandra Ho    |
| Rugby           | Zach Teo     |
| Badminton       | Oh Sim       |
| Student Council | Hong Tee     |

StudentCCA

| MatricNo | CCAName         |
|----------|-----------------|
| 1        | Tennis          |
| 2        | Choir           |
| 2        | AVA             |
| 3        | Rugby           |
| 4        | Badminton       |
| 5        | Choir           |
| 5        | Student Council |

The primary key for table Student will be MatricNo.

MatricNo and CCAName will form a composite key for table StudentCCA.

CCAName will be the primary key for table CCAInfo.

The tables in standard notation are

Student (MatricNo, Name, Gender, CivicsClass, CivicsTutor, HomeRoom)

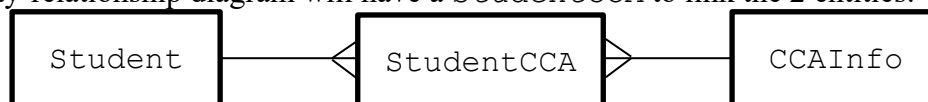
StudentCCA (MatricNo, CCAName)

CCAInfo (CCAName, CCATeacherIC)

The **primary key** for each table is indicated by underlining one or more attributes.

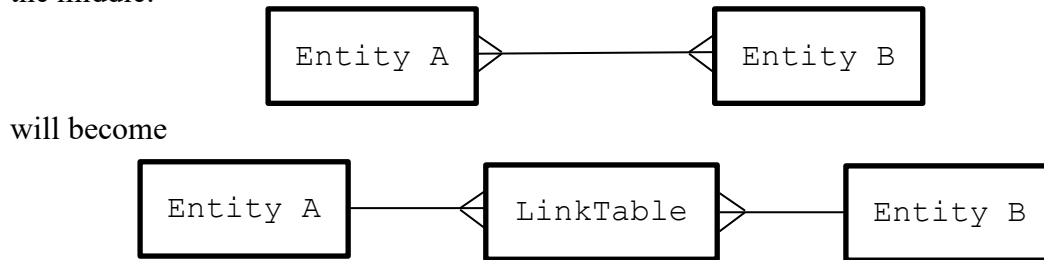
**Foreign keys** are indicated by using a dashed underline.

The entity-relationship diagram will have a StudentCCA to link the 2 entities:



### Dealing with a many-to-many relationship

Whenever two entities have a many-to-many relationship, a link table is always needed in the middle:



### 6.3 Third Normal Form

For a table to be in 3NF, it must satisfy two conditions:

- 1) The table should be in 2NF.
- 2) The table should not have transitive dependencies.

**A table in 3NF contains no non-key dependencies.**

Looking at table Student, CivicsTutor and HomeRoom are dependent on CivicsClass, and CivicsClass is dependent on MatricNo. Therefore, CivicsTutor and HomeRoom are transitively dependent on MatricNo. To remove the transitive dependency, we decompose the Student table as follows:

Student

| MatricNo | Name              | Gender | CivicsClass |
|----------|-------------------|--------|-------------|
| 1        | Aisyah Bte Rahman | Female | 28S28       |
| 2        | Cheryl Lim        | Female | 28S28       |
| 3        | Arjun Singh       | Male   | 28S30       |
| 4        | Benjamin Ong      | Male   | 28S30       |
| 5        | Huda Ismail       | Female | 28S30       |

Civics

| CivicsClass | CivicsTutor | HomeRoom |
|-------------|-------------|----------|
| 28S28       | Paul Poh    | A33      |
| 28S30       | David Lee   | B55      |

MatricNo remains the primary key for table Student.

CivicsClass will be the primary key of the newly formed table Civics.

The final design after normalisation is represented below:

Civics (CivicsClass, CivicsTutor, HomeRoom)  
 Student (MatricNo, Name, Gender, CivicsClass)  
 StudentCCA (MatricNo, CCAName)  
 CCAInfo (CCAName, CCATeacherIC)

The **primary key** for each table is indicated by underlining one or more attributes.

**Foreign keys** are indicated by using a dashed underline.

This design could also be represented using an entity-relationship diagram.

